SMART CAMPUS ANALYTICS USING PYTHON HADOOP STREAMING

Hadoop Run and Viva Guide

Status as of 18 October 2026 | Repository: SmartCampus (branch main)

Where the project stands

All six required MapReduce jobs and the extension Job 7 are implemented, tested, validated locally and **verified on real Hadoop 3.5.0**. The completed worksheet, screenshots and results dashboard are in the repository. This guide explains how to reproduce the Hadoop run and what to be ready to explain in the viva.

Status	What it covers
A. Verified locally	All seven jobs. 38 automated tests pass (0 failed, 0 skipped). 22 validation checks pass. Every output row matches an independent reference implementation. Results are identical with and without the combiner and with 1-4 reducers.
B. Verified on Hadoop	**Hadoop 3.5.0, standalone mode, OpenJDK 17.** On 11 Oct 2026 all nine stages of Jobs 1-6 matched the local results; on 18 Oct 2026 all ten stages, including the map-only Job 7, matched. `-files`, the combiner, the Job 6 partitioner and both distributed-cache side files work. Hadoop appends a trailing TAB to output lines (REVIEW F6).
C. Not yet verified	A multi-node cluster or pseudo-distributed HDFS/YARN setup, and more than one reducer on real Hadoop. Optional for this lab.

Before you start. Open a terminal in the project folder. Every command in this guide assumes you are there:
`cd "/Users/deepesh/Developer/Big Data project"`

1. Run the jobs on real Hadoop

Hadoop's **standalone mode** runs the real Hadoop Streaming engine on your laptop against local files. It needs no cluster and no HDFS, and it tests exactly what the local simulator cannot: Hadoop's own key sorting, shipping scripts with `-files`, the distributed-cache side files of Jobs 5 and 7, the Job 6 partitioner, a map-only job, and Hadoop's output format.

Step 1.1: Install Hadoop

Homebrew's Hadoop package installs OpenJDK 17 as a dependency and points Hadoop at it, so no `JAVA_HOME` setup is needed:

```
brew install hadoop
hadoop version                  # must print "Hadoop 3.x.y"
```

A warning such as *Unable to load native-hadoop library* is normal on macOS. Ignore it.

Step 1.2: Refresh the local results

The Hadoop results are compared with `outputs/`, so make sure those are current:

```
python3 scripts/generate_data.py             # seed 42, identical to the committed data
python3 scripts/run_pipeline.py --run-tests  # expect 22 [PASS] lines, exit code 0
```

Step 1.3: Run all seven jobs on Hadoop

One script runs all ten Hadoop stages (Job 4 has two, Job 5 three), with the combiner, the Job 6 partitioner and the map-only Job 7, and then compares every result with the local output:

```
scripts/run_hadoop.sh                # writes to /tmp/smartcampus_hadoop
```

Expected result (about 20 seconds to 2 minutes):

```
== job1
...
== job7

== Comparison with local outputs/ (trailing TAB stripped)
job1 MATCH (113 lines incl. header)
job2 MATCH (13 lines incl. header)
job3 MATCH (13 lines incl. header)
job4 MATCH (67 lines incl. header)
job5 MATCH (7 lines incl. header)
job6 MATCH (785 lines incl. header)
job7 MATCH (67 lines incl. header)
Trailing TAB on reducer output lines: YES (review finding F6 confirmed)
```

Success criterion. All seven lines say **MATCH** and the script exits with code 0 (`echo $?` prints 0). A summary is saved to `/tmp/smartcampus_hadoop/compare.txt`, and per-job Hadoop logs, including the SmartCampus rejection counters, are in `/tmp/smartcampus_hadoop/logs/`. The project folder name contains spaces, which Hadoop cannot handle in paths, so the script copies `src/` and `data/` into a staging folder first.

Step 1.4: If something fails

Symptom	Cause and fix
`hadoop not found on PATH`	Hadoop is not installed or the shell has not reloaded. Run `brew install hadoop` and open a new terminal.
`JAVA_HOME is not set`	Hadoop was not installed through Homebrew. Point it at a Java 17 JDK, for example `export JAVA_HOME="$(brew --prefix openjdk@17)"`.
`hadoop-streaming jar not found`	Locate it with `find $(brew --prefix hadoop) -name 'hadoop-streaming-*.jar'`, then run `HADOOP_STREAMING_JAR=<path> scripts/run_hadoop.sh`.
Job fails with `python3: not found` or exit code 127	The Hadoop task cannot see your Python. Run `PYTHON=$(which python3) scripts/run_hadoop.sh`.
Job fails with `PipeMapRed.waitOutputThreads(): subprocess failed`	A Python script crashed inside Hadoop. The traceback is in the job's log under `/tmp/smartcampus_hadoop/logs/`.
A job says **DIFFERS**	Open `/tmp/smartcampus_hadoop/logs/jobN.diff`. Lines starting with < came from Hadoop and lines starting with > from the local run.

Optional: a pseudo-distributed (HDFS) run. Standalone mode proves Streaming compatibility. A single-node HDFS setup additionally shows the `hdfs dfs -put / -cat` workflow. Follow the Apache guide *Hadoop: Setting up a Single Node Cluster*, then use the manual commands in the README's Hadoop section with `/campus/in` paths.

2. Prepare for the viva

Read `docs/WORKSHEET.md` (answers), `docs/ARCHITECTURE.md` (design) and `docs/REVIEW.md` (defects found). Then make sure you can explain these points without notes. They are the parts examiners are most likely to probe.

Topic	What to be able to say
The reference reducer was wrong	Hadoop delivers one `key<TAB>value` line per record, sorted by key. It never joins values with ';'. The worksheet's reducer also reset its totals on every line. On real data it printed one line for the whole dataset instead of 112 hourly rows (REVIEW F1).
How reducers find key boundaries	`itertools.groupby` over sorted lines: when the key changes, the previous group is complete, so memory stays O(1) per key.
Why every value carries a count	Averages are computed from (sum, count). A combiner can then pre-add partial results safely, any number of times. An average of averages would be wrong.
Job 4: two stages	Stage 1 groups by TransactionID, takes the distinct items, sorts them and emits each pair once. Stage 2 counts pairs across transactions. Sorting makes (Coffee, Sandwich) and (Sandwich, Coffee) the same key.
Job 5: passing the global average	Jobs share no memory. The global stage writes one line to a file, and the filter stage reads it (locally via GLOBAL_STATS, on Hadoop via the distributed cache). The comparison is strictly greater-than and exact: 2 x total x G_visits > 3 x G_total x visits.
Job 6: why `RouteID|Hour`	Hadoop's default key is the text before the first TAB. With `RouteID<TAB>Hour`, Hadoop would group by route only and break the averages. A single field joined with `|` is correct by default (REVIEW F2).
Job 7: a map-only join	Each pair is independent, so the job has zero reducers (`-reducer NONE`). The mapper reads Job 4's pair counts and looks up Job 3's unit prices from a side file. Frequency and value rank pairs differently: Coffee + Sandwich earns about 3x Samosa + Tea.
Malformed records	Each row is validated, rejected rows are counted by reason as Hadoop counters, and records = valid + rejected is checked.
How correctness was proven	An independent reference implementation, 38 tests, mutation testing (7 deliberate bugs, all caught), identical results across combiner and reducer settings, and all seven outputs matching on real Hadoop.

Practice questions (worksheet section 10)

Answer each one aloud in under a minute. Short answers are in `docs/WORKSHEET.md`.

☐	What are the three major phases of MapReduce? Why does Hadoop sort mapper output?
☐	What is a combiner, and when is it safe? Which of our jobs use one?
☐	What is a partitioner? Describe the custom partitioner for Job 6.
☐	Can a job have zero reducers? (Yes: Job 7.) What happens with multiple reducers?
☐	What is a composite key? Give the Job 6 example.
☐	What is the effect of an excessively large key group? How does data skew show up here?
☐	How would you test a mapper independently? (Show: cat file | python3 mapper.py)
☐	How would the solution change for millions of records? (Cite the 1M-row benchmark: the combiner cut one map task's shuffle from 249,981 to 119 records.)

Live demo you should be able to perform

```
cat data/wifi_logs.csv | python3 src/job1_wifi_mapper.py 2>/dev/null | head -3
cat data/wifi_logs.csv | python3 src/job1_wifi_mapper.py 2>/dev/null \
    | LC_ALL=C sort | python3 src/job1_wifi_reducer.py | head -3
python3 scripts/run_pipeline.py --combiner --reducers 3   # same results, different execution
```

3. Results dashboard

The `dashboard/` folder is a Next.js 16 site that presents every job's result, design and source code. It reads `outputs/` and `src/` when it is built, so rerun the pipeline before rebuilding it. It is deployed on Vercel (Root Directory `dashboard`, Next.js preset); every push to `main` redeploys it.

```
cd dashboard && npm ci && npm run dev      # http://localhost:3000
npm run build                              # static export to dashboard/out/
```

4. Command reference

Purpose	Command
Regenerate datasets	`python3 scripts/generate_data.py [--seed N]`
Run all jobs locally	`python3 scripts/run_pipeline.py --run-tests`
Extension-challenge mode	`python3 scripts/run_pipeline.py --combiner --map-tasks 4 --reducers 3`
Run the tests	`python3 -m unittest discover -s tests -v`
Run on Hadoop and compare	`scripts/run_hadoop.sh`
Validation report	`outputs/validation_report.json`
Dashboard locally	`cd dashboard && npm run dev`

If a command's output differs from this guide, trust what the command actually prints, and record the difference rather than hiding it.